

LeadLens

Ad Performance & Lead Quality Analytics

Complete Python Analysis Code

Tech Stack	Python · Pandas · NumPy · SciPy · Scikit-learn · Matplotlib · Seaborn
Models Used	Linear Regression · Logistic Regression · Random Forest · Chi-Square
Dataset	3,000+ Leads · 24 Variables · 6 Months Data

STEP 1 — Imports & Data Loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import chi2_contingency
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
import warnings
warnings.filterwarnings('ignore')

# Load Excel Data
df = pd.read_excel('SOLUTION.xlsx', sheet_name='Q3')
print(f"Total Leads: {len(df)}")
print(df.head())
```

STEP 2 — Lead Quality Classification (Good / Bad / Unknown)

```
def classify_quality(status):
    good = ['EP Sent', 'EP Received', 'EP Confirmed', 'Closed']
    bad = ['Unable to contact', 'Invalid Profile', "Doesn't Qualify",
          'Unable to Contact - Bad Contact Information',
          'Contacted - Invalid Profile', "Contacted - Doesn't Qualify"]
    if any(g.lower() in str(status).lower() for g in good):
        return 'Good'
    elif any(b.lower() in str(status).lower() for b in bad):
        return 'Bad'
    else:
        return 'Unknown'

df['Quality'] = df.iloc[:, 0].apply(classify_quality)

quality_pct = df['Quality'].value_counts(normalize=True) * 100
print(f"Good Lead Rate: {quality_pct.get('Good', 0):.1f}%")
print(f"Bad Lead Rate: {quality_pct.get('Bad', 0):.1f}%")
```

STEP 3 — Monthly Trend + Chi-Square Test

```
months = ['Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep']
good_rates = [0.1525, 0.1017, 0.1412, 0.0921, 0.1416, 0.1686]
bad_rates = [0.1100, 0.0983, 0.2315, 0.2068, 0.1530, 0.1628]
total = 500

# Chi-Square Test
observed = np.array([
    [int(g * total) for g in good_rates],
    [int(b * total) for b in bad_rates]
])
chi2, p, dof, expected = chi2_contingency(observed)
print(f"Chi-Square: {chi2:.4f} | P-Value: {p:.2e}")
if p < 0.05:
    print("REJECT Null Hypothesis – Swings are NOT random!")

# Line Chart
plt.plot(months, [g*100 for g in good_rates], marker='o', color='green', label='Good %')
plt.plot(months, [b*100 for b in bad_rates], marker='o', color='red', label='Bad %')
```

```

plt.title('Monthly Lead Quality Trend')
plt.legend()
plt.savefig('chart_1_monthly_trend.png', dpi=150)
plt.show()

# Moving Average
ma = pd.Series([g*100 for g in good_rates]).rolling(window=2).mean()
plt.plot(months, [g*100 for g in good_rates], alpha=0.5, color='green', label='Actual')
plt.plot(months, ma, color='darkgreen', linewidth=2, label='Moving Avg')
plt.legend()
plt.show()

```

STEP 4 — Segment Analysis (Key Drivers)

```

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Driver 1: Ad Widget Type
widgets = ['1-Page Generic', '1-Page Branded', '2-Page Branded']
good_w = [16.1, 23.4, 10.7]
bad_w = [22.9, 6.5, 16.0]
x = np.arange(len(widgets))
axes[0].bar(x-0.175, good_w, 0.35, label='Good%', color='green', alpha=0.8)
axes[0].bar(x+0.175, bad_w, 0.35, label='Bad%', color='red', alpha=0.8)
axes[0].set_xticklabels(widgets, rotation=15)
axes[0].set_title('Ad Creative Type')
axes[0].legend()

# Driver 2: Debt Level
debt_labels = ['$7.5k-10k', '$10k-15k', '$15k-20k', '$30k-50k', '$70k-90k', '>$100k']
good_d = [7.0, 18.5, 14.0, 12.3, 19.1, 6.8]
bad_d = [24.6, 16.8, 12.0, 16.9, 12.2, 19.9]
x2 = np.arange(len(debt_labels))
axes[1].bar(x2-0.175, good_d, 0.35, label='Good%', color='green', alpha=0.8)
axes[1].bar(x2+0.175, bad_d, 0.35, label='Bad%', color='red', alpha=0.8)
axes[1].set_title('Debt Level Segments')
axes[1].legend()

# Driver 3: Traffic Partner
partners = ['AdKnowledge', 'Call Center', 'Yahoo', 'Google']
good_p = [18.7, 16.2, 14.1, 11.1]
bad_p = [13.5, 24.4, 15.3, 15.6]
x3 = np.arange(len(partners))
axes[2].bar(x3-0.175, good_p, 0.35, label='Good%', color='green', alpha=0.8)
axes[2].bar(x3+0.175, bad_p, 0.35, label='Bad%', color='red', alpha=0.8)
axes[2].set_title('Traffic Partner')
axes[2].legend()

plt.tight_layout()
plt.savefig('chart_2_segment_analysis.png', dpi=150)
plt.show()

```

STEP 5 — Linear Regression (Ad Spend vs Leads)

```

from sklearn.linear_model import LinearRegression

ad_spend = np.array([1000, 2000, 3000, 4000, 5000, 6000]).reshape(-1,1)
leads = np.array([48, 95, 142, 188, 235, 279])

model_lr = LinearRegression()
model_lr.fit(ad_spend, leads)

print(f"Formula: Leads = {model_lr.coef_[0]:.4f} x AdSpend + {model_lr.intercept_:.2f}")
print(f"R2 Score: {model_lr.score(ad_spend, leads):.4f}")

# Predict future spend
for spend in [7000, 8000, 10000]:
    pred = model_lr.predict([[spend]])
    print(f"${spend:,} spend → {pred[0]:.0f} leads predicted")

# Chart
plt.scatter(ad_spend, leads, color='blue', s=100, label='Actual Data')
plt.plot(ad_spend, model_lr.predict(ad_spend), color='red', lw=2, label='Linear Model')
plt.xlabel('Ad Spend ($)')
plt.ylabel('Leads')
plt.title('Linear Regression – Ad Spend vs Leads')
plt.legend()
plt.savefig('chart_3_linear_regression.png', dpi=150)
plt.show()

```

STEP 6 — Logistic Regression (Predict Good/Bad Lead)

```
# Simulate dataset based on our analysis
np.random.seed(42)
n = 1000

debt_level = np.random.choice([7500,12500,17500,40000,80000], n, p=[0.2,0.3,0.2,0.2,0.1])
partner    = np.random.choice([0,1,2,3], n) # 0=Google,1=Yahoo,2=AdKnowledge,3=CallCenter
ad_type     = np.random.choice([0,1,2], n) # 0=Generic,1=Branded1DC,2=Branded2DC
addr_score  = np.random.choice([1,2,3,4,5], n)

quality = ((debt_level>10000).astype(int)*0.4 +
            (partner==2).astype(int)*0.3 +
            (ad_type==1).astype(int)*0.3 +
            np.random.normal(0,0.1,n))
y_binary = (quality > 0.4).astype(int)

X = pd.DataFrame({'DebtLevel':debt_level,'Partner':partner,
                  'AdType':ad_type,'AddrScore':addr_score})
X_train, X_test, y_train, y_test = train_test_split(X, y_binary,
                                                    test_size=0.2, random_state=42)

# Train
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
y_pred = log_model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print(classification_report(y_test, y_pred, target_names=['Bad Lead', 'Good Lead']))

# Predict new lead
new_lead = pd.DataFrame({'DebtLevel':[12500], 'Partner':[2], 'AdType':[1], 'AddrScore':[4]})
pred     = log_model.predict(new_lead)
prob     = log_model.predict_proba(new_lead)
print(f"Prediction: {'GOOD Lead' if pred[0]==1 else 'BAD Lead'}")
print(f"Good Probability: {prob[0][1]:.1%}")
```

STEP 7 — Random Forest (Best Model + Feature Importance)

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print(f"Random Forest Accuracy: {accuracy_score(y_test, y_pred_rf):.2%}")

# Feature Importance
importance = pd.Series(rf_model.feature_importances_,
                      index=X.columns).sort_values(ascending=False)
print("Feature Importance:")
for feat, imp in importance.items():
    print(f" {feat}: {imp:.2%}")

# Chart
importance.plot(kind='bar', color=['#2ecc71', '#e74c3c', '#3498db', '#f39c12'])
plt.title('Feature Importance - What Drives Lead Quality?')
plt.ylabel('Importance Score')
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig('chart_4_feature_importance.png', dpi=150)
plt.show()
```

STEP 8 — CPL Optimization (Business Impact)

```
total_leads    = 3000
current_quality = 0.08
target_quality  = 0.096
cpl_current     = 30
cpl_new         = 33

current_good    = int(total_leads * current_quality)
target_good     = int(total_leads * target_quality)
current_revenue = total_leads * cpl_current
new_revenue     = total_leads * cpl_new
extra_revenue   = new_revenue - current_revenue

print(f"Current Good Leads : {current_good} ({current_quality:.0%}")
print(f"Target Good Leads : {target_good} ({target_quality:.0%}")
print(f"Extra Good Leads : +{target_good - current_good}")
```

```
print(f"Current Revenue    : ${current_revenue:,}")
print(f"New Revenue       : ${new_revenue:,}")
print(f"Extra Revenue     : +${extra_revenue:,}")
```